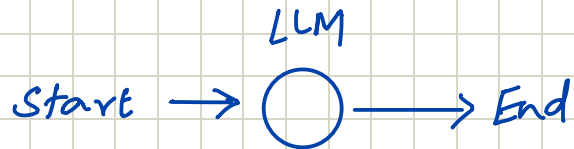


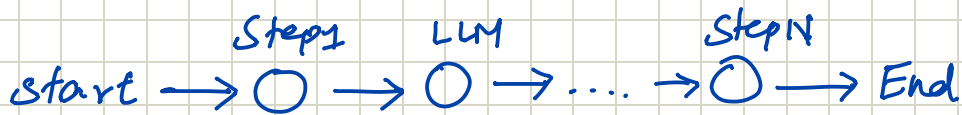
Langchain & LangGraph

A solitary language model is fairly limited.



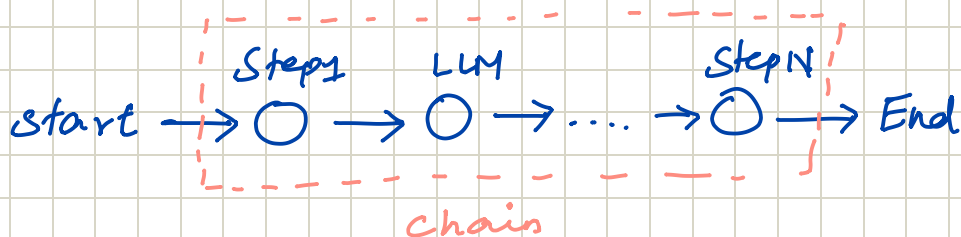
It does not have access to external context, tools, multi-step workflows.

So many LLM applications use a control flow with



steps pre/post LLM calls. It can be tools calls, retrieval etc.

This control flow forms a chain

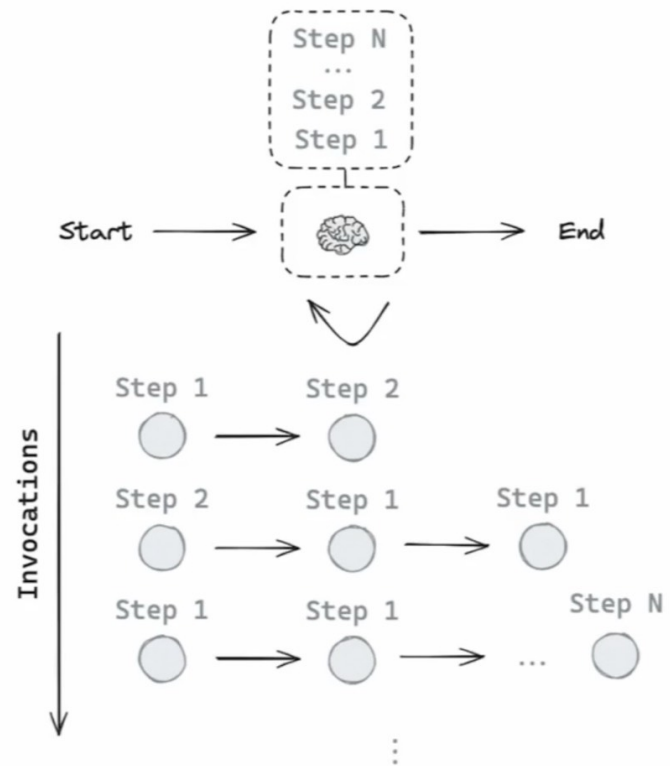


chains are very reliable because same control flow everytime. Eg: for every invocations, same steps followed.

But, we want LLM systems that can pick their control flow.

Depending upon the problem, LLM decides which steps to be carried out during runtime.

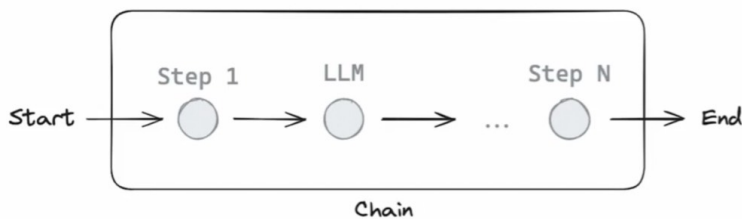
Agent $\approx$ Control flow defined by the LLM



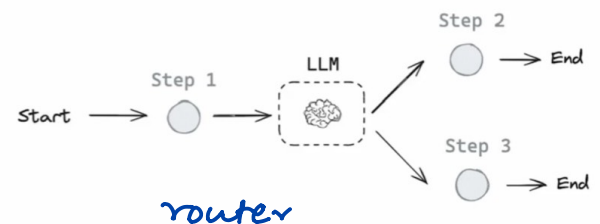
Fixed

VS

LLM defined control flow.

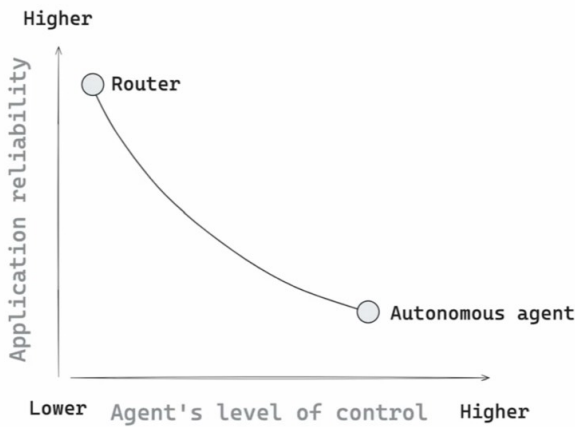


VS



Many kind of agents like Router, full Autonomous agents. Routers - where LLM controls a single step in a flow & may choose between narrow set of options

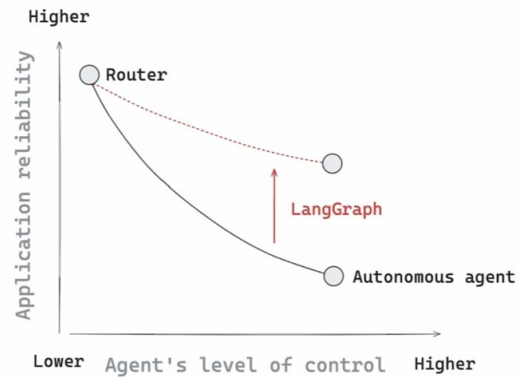
But, practical challenges.



The higher the control you give the LLM, the application reliability becomes lower. So this is the motivation for developing the LangGraph, which helps in bending the reliability curve.

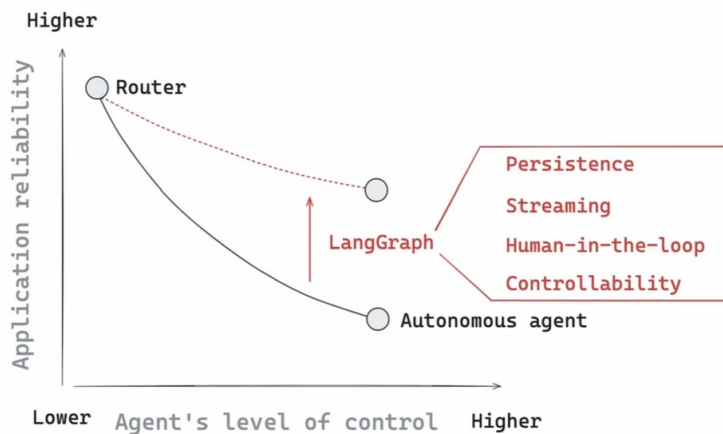
Langgraph helps you to build agents that maintain reliability, even as you push out the level of control.

LangGraph helps you bend the reliability curve



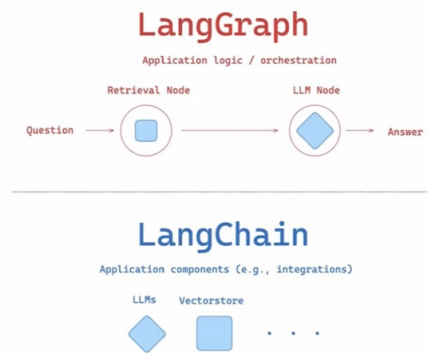
LangGraph has advanced features/pillars.

Pillars



LangGraph is more about orchestration / application logic where LangChain is more of application components

LangGraph  LangChain



State & Memory.

State Schema:

When we define LangGraph StateGraph, we use a state schema.

The state schema represents the structure and types of data that our graph use. All the nodes are expected to communicate with the schema.

LangGraph offers flexibility in how you describe the state schema, accommodating various python types & validation approaches.

Example: TypedDict, Literal, Dataclass (no enforcement

Pydantic: * powerful data validation & settings management library using python type annotations.
* well suited for defining state schemas in langgraph due to its robust validation capabilities

on types on runtime)
↳ assign invalid values which don't raise any errors.

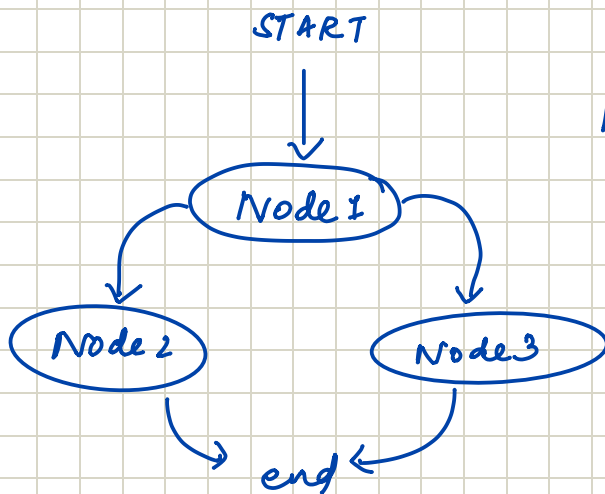
* It can perform validation to check the data conforms to specified types and constraints during run time.

State Reducers:

We read about different state schemas like TypedDict, Dataclasses & Pydantic.

Now we are going to learn how state updates are performed on specific keys/channels in the state schema.

Problem:



here updating
states in node 2
& node 3 becomes
"ambiguous"

Reducers eliminates
this problem by
using "annotated."

eg

```
class State(TypedDict):  
    foo: Annotated[  
        list[int], add  
    ]  
        ↓  
    reducer  
    function
```

We can also create custom Reducers; like if in
add, we get "None" as input, we get error. we can create
custom reducers to handle this case.

Messages: If we want to work with messages, we use
MessageState which has add-messages reducer.
we can rewrite the messages using id & also we
can delete the messages with id of specific message.

Multiple schemas:

W.K.T state schemas & reducer help with communication.

There are cases where we want multiple schemas. Let's say input & output may have different schemas. Or internal nodes may pass information that is not needed in output/input.

Private state:

Useful for anything that needed for intermediate work logic but not relevant for overall graph input/output. It's more like "private" keyword in inheritance where the user will not know intermediate work.

Overall State, Input State, Output State.

Overall state will have all parameters, where the inputState & OutputState will have some of OverallState & apply filters on StateGraph.

Eg:

graph = StateGraph (OverallState, input = InputState, output = OutputState)

MessageState

Messages: TrimMessages & filterMessage built ins can be used for token length & Latency Reduction.

Memory: Memory is retained across graph & after certain count of messages we summarized & deleted some message for less token usage.
 → using checkpoint & threads

Human in the Loop:

Human in the loop which builds on Memory and helps to interact with the agents.

LangGraph is built with first class support for streaming.

Streaming:

Streaming full state:

It has `.stream` and `.astream` are sync methods for streaming back results. Langgraph supports different streaming modes for graph state.

values: This streams the full state of graph after each node is called.

updates: This stream updates to the state of the graph after each node.

Streaming tokens:

In particular LLM calls, it is common to stream the tokens as they are generated. We can do use `.astream_events` methods, which streams back events as they happen inside nodes.

Each event with a dict: `event`, `name`, `data`, `metadata`.

New Streaming Method only works for APIs is Message mode

Breakpoints

The motivation for human in the loop.

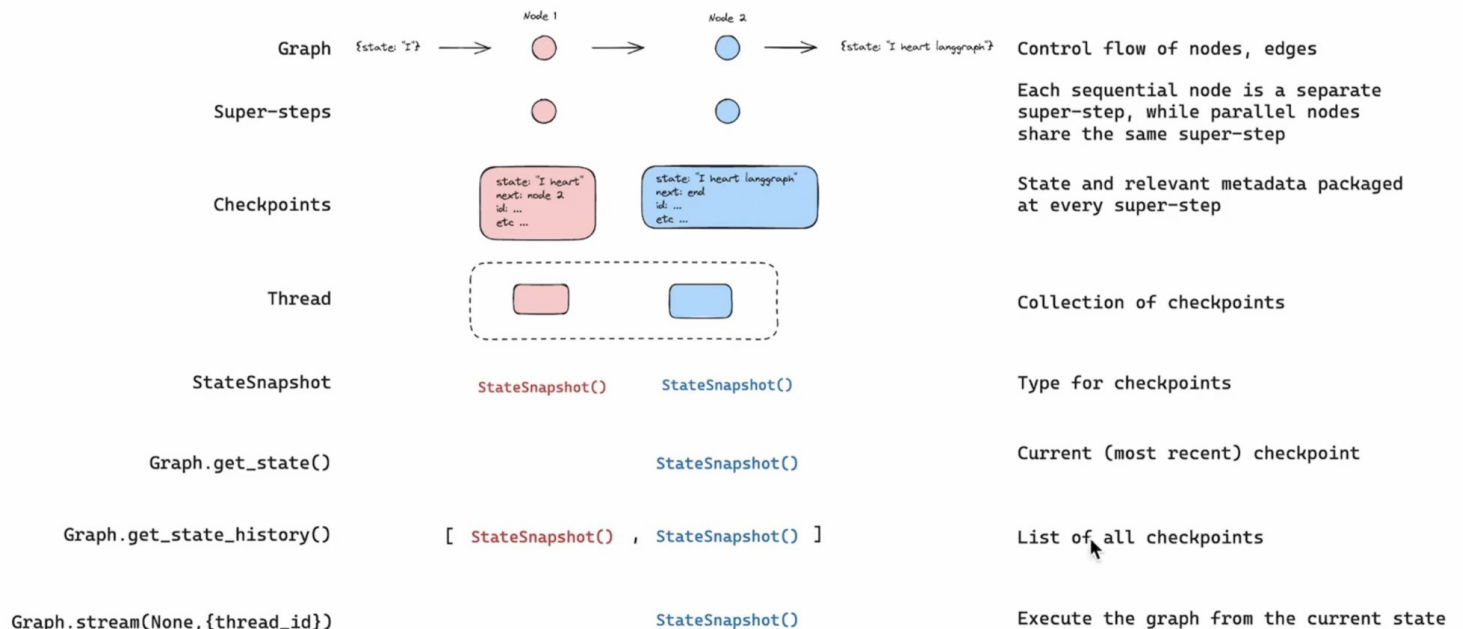
(i) Approval (ii) Debugging (iii) Editing

Approval:

LangGraph offers several ways to get or update agent state systematically to support human in the loop workflows. We introduce "breakpoints" which provides simple way to stop the graph at specific steps.

Ex: If we want to approve the agent to use it for any tool, simply we can use interrupt-before=["tools"]

When we invoke the graph with `None`, it will just continue from the last state checkpoint!



We can also use `interrupt-before=["tools"]` using API as well. to the stream method

Editing Graph State:

We can use breakpoints to edit the graph state as well.

ex: `interrupt_before = ["assistant"]`. Interrupt done.

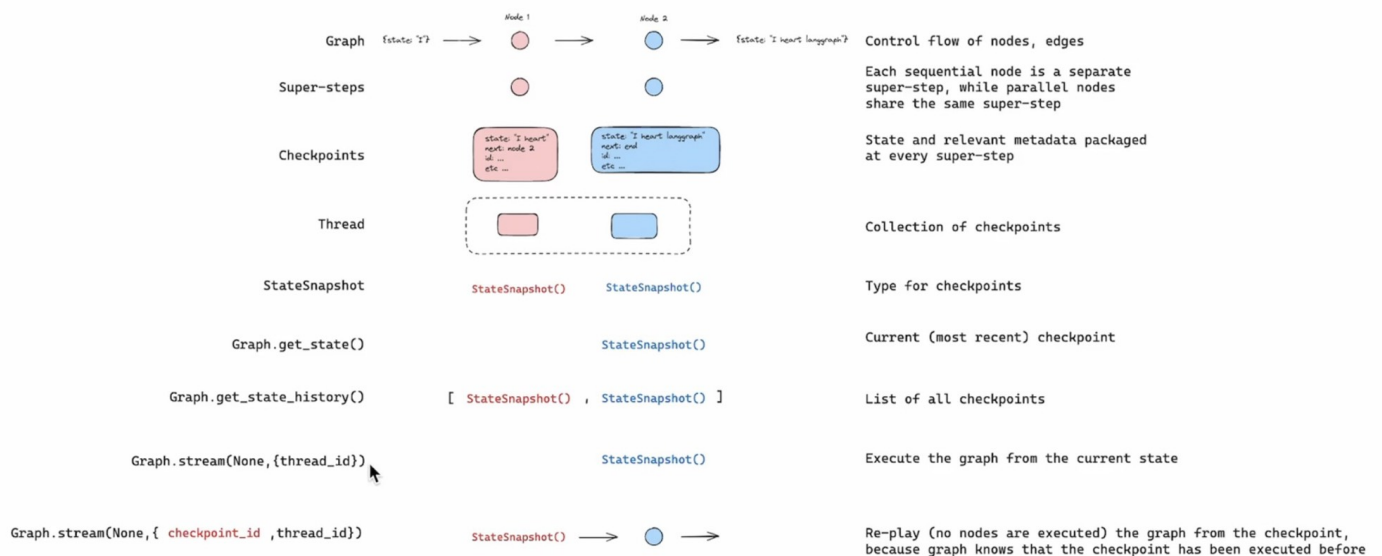
Now `graph.update_state(thread, {'messages': [HumanMessage("", "")]})`

Debugging:

Replaying

Replaying

We can re-run our agent from any of the prior steps.



Let's look back at the step that recieved human input!

here `graph.stream(None, [checkpoint_id, thread_id])` helps to execute the state belongs to checkpoint. It helps to debug or replay what happened during runtime.

`Graph.get_state_history()` helps to get list of all checkpoints.

To replay, get the `StateSnapshot` & have config.

Then stream the event with config to replay it again

Forking:

If we want to run from same step but with different input, this is forking.

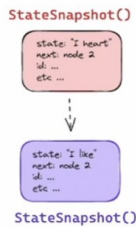
What if we want to run from that same step, but with a different input! This is forking.

```
Graph.update_state({thread_id},{state: "I like cold brew"})
```



Forks the current checkpoint, creating a new one with new state

```
Graph.update_state({ checkpoint_id ,thread_id}  
{state: "I like"})
```

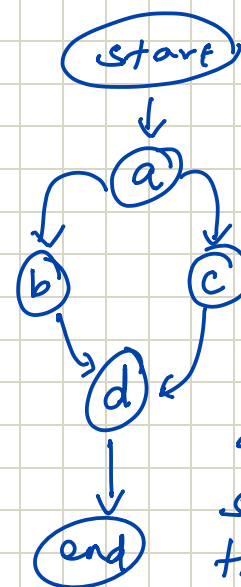


Forks the specified checkpoint, creating a new one with new state

```
Graph.stream(None,{checkpoint_id ,thread_id})
```

Re-invokes the graph because it knows that this has not been executed

Parallelization:



"b" and "c" are writing the same state same time. This causes an error.

"StateReducers resolves this error".

Subgraphs:

Subgraphs allow you to create and manage different states in different parts of the graph.

Communication is done with non overlapping keys

* The subgraphs can access docs from the parent

* The parent can access state of sub graphs.

Map Reduce:

It is essential for efficient task decomposition & parallel processing.

2 phases:

1.) Map - Break a task into smaller subtasks, processing each subtask in parallel.

2.) Reduce - Aggregate the results across all of the completed subtasks.

Memory:

Memory is the cognitive function that allows people to store, retrieve & use information to understand their present & future.

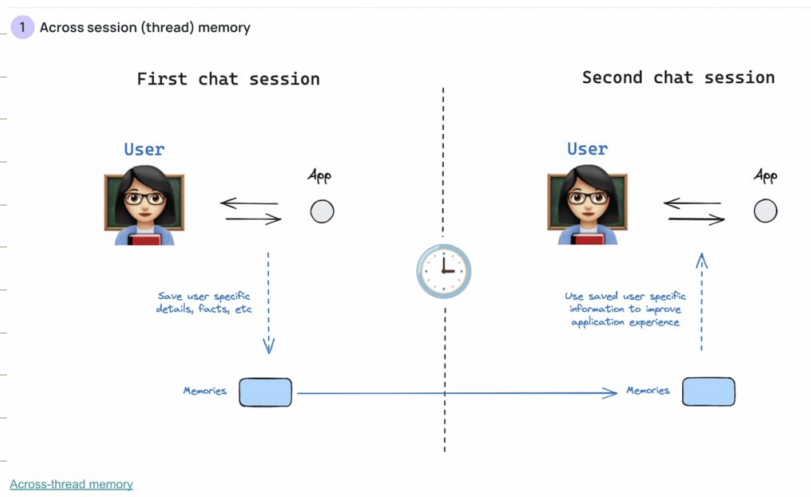
1.) Within session (thread) Memory:

A thread in LangGraph store between AI bot &

user within single chat session. This thread contains the entire history.

2.) Across The session (Thread) Memory:

This is the idea that some information can be persisted.



Comparison Within Vs Across Session

	Short-term	Long-term
Scope	Within session (thread)	Across session (thread)
Example use-case	Persist conversational history, allow interruptions in a chat (e.g., if user is idle or to allow human-in-the-loop)	Remember information about a specific user across all chat sessions
LangGraph usage	Checkpoint	Store

Short term Memory : Can be summarized (or) filtered.

Long term Memory : Semantic
(Agents)
facts about a user

Episodic
(memories)
Past agent actions

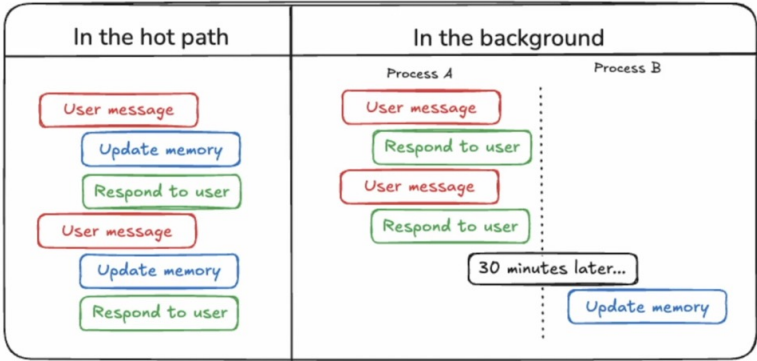
Procedural
(Instructions)
Agents system prompt.

Semantic (Profile) → about a user profile
 Semantic (Collection) → Eg: Collection of ToDoList.

	Type	Pro	Con
Profile	Single document	Easily retrieved single representation	Challenging to maintain as it grows larger
List	List of documents	Allows for smaller, narrowly scoped memories w/ easy addition of new information	Retrieval can be challenge as the list grows larger

Episodic (few shot) - Prior reasoning trajectories

When do you update Long term Memory updating?



	Type	Pro	Con
Hot-path	During runtime (ChatGPT)	Real-time updates with transparency for user	Can affect UX / latency and degrade performance
Background	As a separate process	Lower risk of UX / performance degradation	Frequency of memory writing needs to be tuned

LangGraph Store (Long term Memory)

LangGraph Memory Store as a way to save & retrieve information across threads in LangGraph.

